

Experiment 20 — The coverage sweep: every numeric builtin against NumPy/SciPy

Mathilda — execution-speed experiments

Contents

Experiment 20 — The coverage sweep: every numeric builtin against NumPy/SciPy	1
Why a twentieth experiment, and why it is a different shape	1
The two tools	2
What the static half found	2
The blind spot the static half has	2
Results, by group	2
What was fixed	3
The gap register	5
Two things the harness itself got wrong	5
See also	5

Experiment 20 — The coverage sweep: every numeric builtin against NumPy/SciPy

Date: 2026-07-31 · Code: `tools/numeric_coverage.py`, `tools/numeric_sweep.py`, `src/numloop.c`, `src/funcprog.c`, `src/pack.c`, `src/list/minmax.c`, `src/list/total.c`, `src/linalg/numarray.c` · Result: MinMax 636×, the seven predicate heads 35–51×, Norm 2.5×; a register of 42 measured gaps and 96 absent functions

Common method in [README.md](#).

Why a twentieth experiment, and why it is a different shape

Experiments 1–19 were each driven by a workload: pick a kernel from a real application, measure it, find out why it is slow. That method produced every large win in this tree, and the finding it kept producing was the same one:

An operation had a working buffer path and a working list path, and quietly took the second.

Which is exactly the finding a workload-driven sweep is worst at generalising. It finds those one accident at a time, and only for the heads the chosen workloads happen to touch. Nineteen experiments is a lot of coverage by accident; it is no coverage by construction. Nothing in the tree could answer "which builtins have not been measured?"

So this experiment inverts the method: enumerate every builtin the system registers, and ask of each one whether a numeric argument reaches machine code — first statically, from the registries that decide dispatch, then dynamically, by running it beside the same operation in NumPy/SciPy.

The two tools

tools/numeric_coverage.py — the static half. Extracts every name passed to `syntab_add_builtin` (676) and joins it against the three registries that decide whether a numeric argument reaches machine code: `nd-kernels.c`'s `REG_U/REG_B/REG_N`, `pack.c`'s `AWARE`, and `pack.c`'s `INT64_OK`. A head in none of them, that a numeric workload can reach, is by construction running at one `Expr` node per element.

The judgement the static half cannot make is which heads should care — `Names[]` has no business on a `float64` buffer. That is a hand-assigned category table in the script, the one part worth reviewing; everything else is derived. Unclassified defaults to "symbolic", so a wrong default under-reports rather than inventing work.

tools/numeric_sweep.py — the dynamic half. 283 probes over the numeric surface, each timed in Mathilda and in NumPy/SciPy on the same deterministic data, with a scalar checksum of every result compared between the two systems.

```
python3 tools/numeric_coverage.py --gaps           # registered but with no fast path
python3 tools/numeric_coverage.py --missing        # heads that do not exist at all
python3 tools/numeric_sweep.py --group reduce      # measure one group
python3 tools/numeric_sweep.py --diag             # add a MATHILDA_NO_PACK column
python3 tools/numeric_sweep.py --emit DIR          # write the .m and .py it runs
```

What the static half found

676 builtins, of which 339 are numeric-relevant by the category table:

category	total	has a registered path
numeric	339	194
scalar	12	—
symbolic	299	10
io	26	2

145 numeric heads have no registered fast path. That is an upper bound on the work, not a work list: a head that never reads an element does not need one. The dynamic half is what separates those from the real gaps.

The blind spot the static half has

A head that NumPy has and Mathilda does not never appears in the source scan at all, so "no fast path" and "no function" look identical from the source. They are not the same finding.

The first dynamic run surfaced this as nine statistics rows that read as slow — `Quantile`, `Skewness`, `Kurtosis`, `Correlation`, `Covariance`, `Standardize`, `GeometricMean`, `HarmonicMean`, `TotalVariation`. They are not slow. They return unevaluated. `--missing` now asks the running binary via `Names["*"]`, and 96 of the 436 numeric heads named in the category table are not defined at all, including `CholeskyDecomposition`, `KroneckerProduct`, `MatrixExp`, `Diagonal`, `ArrayReshape`, `Ordering`, `BinCounts`, `SparseArray`, the Chebyshev/Hermite/Laguerre/Jacobi orthogonal polynomials, and the elliptic integrals. That is a coverage gap rather than a speed gap, and it is recorded rather than fixed — this experiment is about whether the functions Mathilda has run at machine speed.

Results, by group

Median and worst ratio to NumPy/SciPy over the comparable probes in each group. 223 of 283 probes have both columns; 31 are unevaluated (above), 2 did not finish.

group	n	median vs NumPy	worst
elementwise	57	1/1.07×	2199×
special	32	1/1.34×	207×
transform	5	1/1.26×	8.2×
reduce	15	1.3×	1470×
linalg	18	1.6×	20924×
scan	8	1.7×	338×
struct	36	3.8×	106841×
functional	12	7.7×	1023×
build	13	26.7×	1679×
integer	8	70.4×	443×
linalg-small	4	84.6×	122×
mask	14	1466×	5221×

The elementwise and special-function surface is done. Its median is at parity with NumPy and better than SciPy, and the individual rows go a long way past that: Exp 4.2× ahead, Cot 6.6×, AiryBi 11.8×, ExpIntegralEi 10.5×, Gamma 4.2×. That is the threaded-kernel work of experiments 2, 5 and 6 paying off across a surface no single workload had ever exercised.

The distance is everywhere else, and it is concentrated in kinds of operation rather than in individual heads — which is what a coverage sweep can see and a workload sweep cannot.

What was fixed

Everything below is verified by `tests/test_pred_compile.c`, whose every assertion evaluates the same source twice — compiled path and interpreter — and requires identical printed output.

#		before	after	
1	MinMax was not on the AWARE list	307 ms	483 μs	636×
2	The seven predicate heads had no compiled path — AllTrue	396 ms	9.90 ms	40×
	AnyTrue	382 ms	9.75 ms	39×
	TakeWhile	315 ms	6.22 ms	51×
	Select	82.6 ms	2.36 ms	35×
	Select with a band ($0.25 < \# < 0.75 \ \&$)	496 ms	21.9 ms	23×
3	na_load_vector copied element by element — Norm	3.95 ms	1.58 ms	2.5×
	Normalize	5.56 ms	3.18 ms	1.7×
4	Total[{}] answered {} where Mathematica gives 0	—	—	correctness

1. MinMax — the recurring defect, found by enumeration

`MinMax[v]` delegates to `Min[v]` and `Max[v]`, each of which has had a buffer path (`ndred_min / ndred_max`) all along. But `MinMax` was not on `pack.c`'s AWARE list, so the transparency gate materialised 10^6 Expr nodes before the builtin saw them, and both delegates then ran the generic List code on the boxed copy. 307 ms, against 255 μs and 232 μs for the same `Min[v]` and `Max[v]` called directly.

This is the fourth appearance of the identical defect — after the 26 linear-algebra heads, Nest, and Fourier — and the first one found by enumerating the registry rather than by a workload tripping over it. That is the argument for this experiment in one row.

Two lines: accept an NDAarray in `builtin_minmax`'s argument test (so the head can be marked aware at all), and add it to AWARE and INT64_OK.

2. The predicate family — the boolean half of `numloop_map`

Seven heads apply a boolean test to every element: `Select`, `AllTrue`, `AnyTrue`, `NoneTrue`, `TakeWhile`, `LengthWhile`, `SelectFirst`. Every one of them opened with `ndstruct_delist_repack` — materialise the buffer, then call the test through the interpreter once per element.

Map has had a compiled path since the auto-compilation experiment (`numloop_map`, experiment 4). The boolean half was never written, and it left these the slowest numeric heads in the system: `AllTrue[v, # > 0 &]` over 10^6 cost 416 ms against `np.all(v > 0)`'s 319 μ s.

`compile_function` cannot be used directly — a comparison is not in the arithmetic subset it compiles, and NumProg's VM has no boolean value at all. So a predicate compiles as a tree of comparisons over `NumProgs`: each comparison operand is wrapped back up as a pure function carrying the original's parameter spec and handed to `compile_function` unchanged, so nothing about the arithmetic subset is duplicated or has to be kept in sync. And, `Or` and `Not` combine the comparisons, short-circuiting in the interpreter's order.

The tolerance is the whole correctness question. Mathilda does not compare machine reals with C's `<`. `compare_numeric` (`src/comparisons.c`) treats two inexact operands as equal when they agree to a relative 2^{-46} , so `Less[1., 1. + 2.^-47]` is `False` where a naive compiled comparison says `True`. That difference is invisible on ordinary data — every other case in the test file passes with or without it — and would have been a silent wrong answer on exactly the near-tie data a numerical program produces. `pred_cmp` reproduces `compare_numeric`'s inexact branch, constant included, and `tolerance_is_reproduced()` in the test pins it (including a check that the premise still holds, so the case cannot quietly become vacuous).

That also fixes the domain: float64 buffers only. An int64 buffer is a list of exact Integers, and `compare_numeric` compares two exact operands through GMP with no tolerance at all — a different rule, which this declines rather than approximates.

The chain was not optional, and the first attempt at it did nothing. `0.25 < # < 0.75` — the ordinary way to write a band, and the shape every windowing predicate takes — does not parse as a three-argument `Less`. It is `Inequality[0.25, Less, Slot[1], Less, 0.75]`, operands and operator symbols alternating. Generalising `Less` to `argc >= 2` compiled cleanly, passed everything, and left the row at 496 ms. `FullForm` said why in one line. The lesson is the tree's own: check what the parser actually built, do not infer it from the surface syntax.

Finally, a comparison operand that reads no variable is folded to a plain double at compile time. `# > 0.5` is the overwhelmingly common shape, and half the VM work in the hot loop was re-deriving `0.5` a million times.

3. `na_load_vector` — the boundary, again

`na_load_vector` mallocs and fills element by element through `ndt_get`, a dtype dispatch per element, for every dtype including float64 — where the bytes it writes are bit-identical to a `memcpy` of the source. Every BLAS bridge entry point, `Norm` and `Normalize` paid for it.

This is exactly the cost performance.md §2 removed from `na_load_matrix` ("converted row-major to column-major one `ndt_get` at a time; it is now a cache-blocked transpose, or a `memcpy` where no transpose is needed"). The vector loader was simply never given the same treatment. `Norm` is 2.5× and `Normalize` 1.7×, and what remains is the 8 MB allocate-and-copy itself: a pure reduction has no reason to copy at all, and a borrow API is the next step.

4. `Total[{}]`

`Total[{}]` answered `{}` — an empty List where a number is expected, which then propagates as a non-number through whatever consumes it. Plus `@@ {}` was already 0. Found because a `Select` that matched nothing fed the sweep's checksum; nothing in the suite covered the empty case because nothing in the suite produced an empty result by accident.

The gap register

What the sweep measured and this experiment did not fix, worst first, so the next one has a work list rather than a hypothesis. Every figure is measured.

#	gap
1	No boolean array dtype. <code>Map[# > 0.5 &, v]</code> builds 10^6 True/False symbols; every mask consumer then walks a box
2	MatrixPower runs an exact/symbolic path on a machine matrix
3	Extract materialises the whole buffer to read three elements
4	The Bessel and ProductLog kernels are not reaching machine code
5	Insert/Delete/Append/Prepend are "correct by omission" in <code>pack.c</code> and box the whole list
6	MemberQ/Count/Position scan a boxed list
7	Array construction: <code>Table[N[i], ...]</code> , <code>Array</code> , <code>Subdivide</code> , <code>Rescale</code> , <code>IdentityMatrix</code> , <code>DiagonalMatrix</code> , <code>UnitVector</code>
8	The integer band, again. <code>Mod/Quotient</code> have registered binary kernels that decline <code>int64</code>
9	Small dense matrices never reach LAPACK
10	N[list] , Chop , Im , Catenate , Sort on <code>int64</code>
11	MatrixExp[A6] on a 6×6 does not complete in 180 s
12	LeastSquares[A500, b500] and PseudoInverse[A300] did not finish in 180 s
13	96 numeric heads do not exist

Two things the harness itself got wrong

Worth recording because both are the method failing, not the system.

A value check tighter than the printing. Mathilda prints six significant figures; NumPy's repr gives seventeen. At a 10^{-6} tolerance the first run reported eleven rows as value mismatches where every one of them agreed — `1.1752e+06` against `1175201.4651842169`. `performance.md` §8 records the identical bug in `hpc_bench.py`. Any comparison against a printed value must be no tighter than the shorter printing.

Probes that encoded the wrong semantics. `Greater` and `EvenQ` are not `Listable` in the Wolfram Language, so `v > 0.5` and `EvenQ[iv]` stay unevaluated on a list — in Mathematica exactly as here. Five probes were timing the evaluator declining to answer and reporting it as a $\sim 3000\times$ gap. They now use the idiomatic spellings, which is what a Wolfram programmer writes and therefore what should be timed. Two more (`Partition[v, 2]`, `Re[v]`) were being compared against a NumPy view, i.e. against a no-op; they now compare against an explicit copy, which is the same computation.

Both were caught by the checksum column. A sweep without one would have shipped both.

See also

- [numeric_coverage_sweep.m](#) — the probe set, runnable unmodified in Mathilda and Mathematica (`tools/numeric_sweep.py --emit`)
- [numeric_coverage_sweep.py](#) — the same algorithms in NumPy/SciPy
- [docs/design/performance.md](#) — the combined table
- [plans/HPC_IMPROVEMENT_PLAN.md](#)